

AUTO SCALING GROUP (ASG)

An Auto Scaling Group (ASG) in Amazon Web Services is a service that automatically manages a group of EC2 instances — it adds or removes instances based on demand.

it integrates with a Load Balancer to distribute traffic evenly across instances, ensuring high availability and performance.

Types of Auto Scaling

- Horizontal scaling (scale out/in) means adding or removing EC2 instances based on demand
- Vertical scaling (scale up/down) means increasing or decreasing the instance size (CPU, RAM)

How It Works

1. You create a **Launch Template**
2. Define an **Auto Scaling Group**
3. Set:

Min / Desired / Max

- Min → Minimum instances always running
- Desired → Current running instances
- Max → Maximum limit

4. Attach a **Scaling Policies**

- Target Tracking (most used)

Example: Keep CPU at 60%

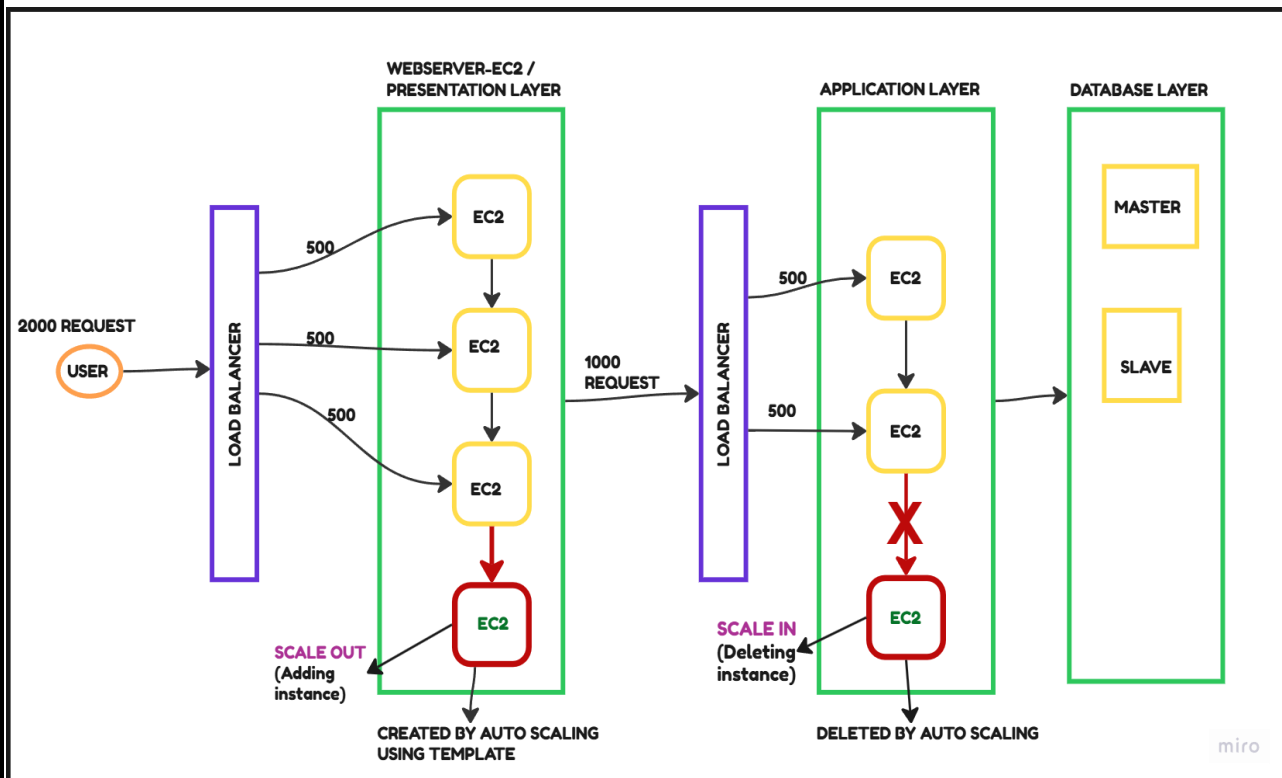
- Simple Scaling

Example: If CPU > 60% → add 1 instance

5. Then ASG will:

-  **Scale Out** → Add instances when load increases
-  **Scale In** → Remove instances when load decreases

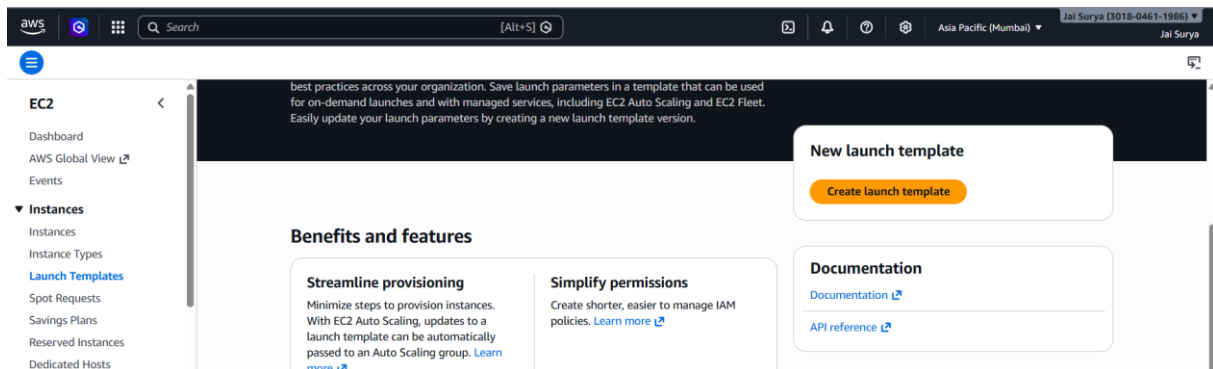
WORKING OF AUTO SCALING GROUP



HANDS-ON FOR AUTO SCALING GROUP

Screenshot: Create Launch Template

- Go to Amazon Web Services
- Click EC2
- Click Launch Templates
- Click Create Launch Template



Screenshot: Enter Template Details

- Enter template name
- Tick provide guidance to help me set up a template

EC2 > Launch templates > Create launch template

Create launch template

Creating a launch template allows you to create a saved instance configuration that can be reused, shared and launched at a later time. Templates can have multiple versions.

Launch template name and description

Launch template name - *required*

my-template

Must be unique to this account. Max 128 chars. No spaces or special characters like '&', '*', '@'.

Template version description

A prod webserver for MyApp

Max 255 chars

[Auto Scaling guidance](#) | [Info](#)

Select this if you intend to use this template with EC2 Auto Scaling

☒ Provide guidance to help me set up a template that I can use with EC2 Auto Scaling

▶ Template tags

▶ Source template

▼ Summary

Software Image

-

Virtual server ty

-

Firewall (securit

-

Storage (volume

-

Cancel

Screenshot: Select AMI

- In Quick Start
- Select Amazon Linux

▼ Application and OS Images (Amazon Machine Image) - required

[Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose **Browse more AMIs**.

Q Search our full catalog including 1000s of application and OS images

Recents

My AMIs

Quick Start

Amazon Linux

aws

macOS

Mac

Ubuntu

ubuntu

Windows

Microsoft

Red Hat

Red Hat

SUSE Linux

SUSE

Debian

debian

Browse more AMIs

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Amazon Linux 2023 kernel-6.1 AMI

ami-0931307dcdc2a28c9 (64-bit (x86), uefi-preferred) / ami-0e6f3e69477999e5d (64-bit (Arm), uefi)

Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible

Screenshot: Configure Instance Settings

- Select instance type
- Select key pair
- Select security group
- Click Create Launch Template

▼ Instance type

Info | Get advice

Advanced

Instance type

t3.micro

Free tier eligible

Family: t3 2 vCPU 1 GiB Memory Current generation: true

On-Demand Linux base pricing: 0.0112 USD per Hour On-Demand SUSE base pricing: 0.0112 USD per Hour

On-Demand Windows base pricing: 0.0204 USD per Hour

On-Demand Ubuntu Pro base pricing: 0.0147 USD per Hour On-Demand RHEL base pricing: 0.04 USD per Hour

▼

☑ All generations

Compare instance types

Additional costs apply for AMIs with pre-installed software

▼ Key pair (login)

Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name

3pmbatch2

▼

↻ Create new key pair

Not applicable for EC2 Auto Scaling

▼ Firewall (security groups)

Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☑ Select existing security group

☐ Create security group

Security groups

Info

Select security groups

▼

↻ Compare security group rules

default sg-02d620cc3320d040d

×

VPC: vpc-00e925f7d875b3505

▼ Advanced network configuration

No network interfaces are currently included in this template. Add a network interface to include it in the launch template.

Add network interface

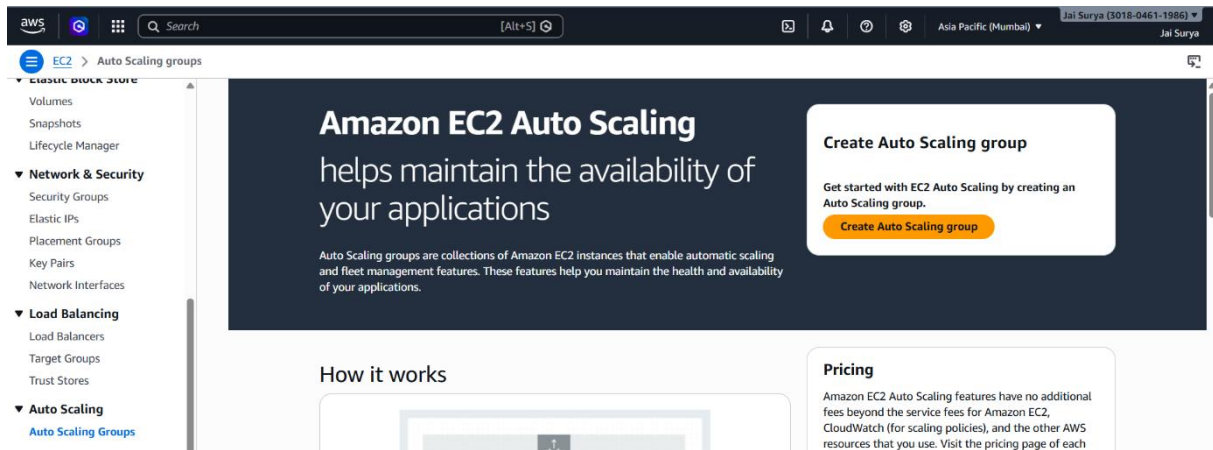
Screenshot: Launch Template Created

- Launch template created successfully

Launch Template ID	Launch Template Name	Default Version	Latest Version	Create Time	Created By
lt-0dbdb5bbb89be22b5	my-template	1	1	2026-03-29T12:08:45.000Z	arn:aws:iam::301804:role/Jai Surya (3018-0461-1986)

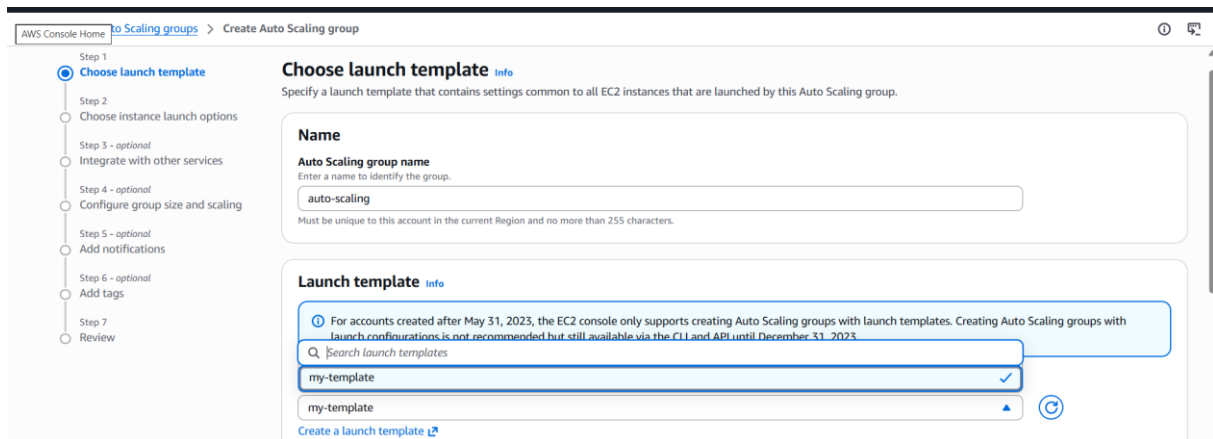
Screenshot: Create Auto Scaling Group

- Go to Amazon Web Services
- Click EC2
- Click Auto Scaling Groups
- Click Create Auto Scaling group



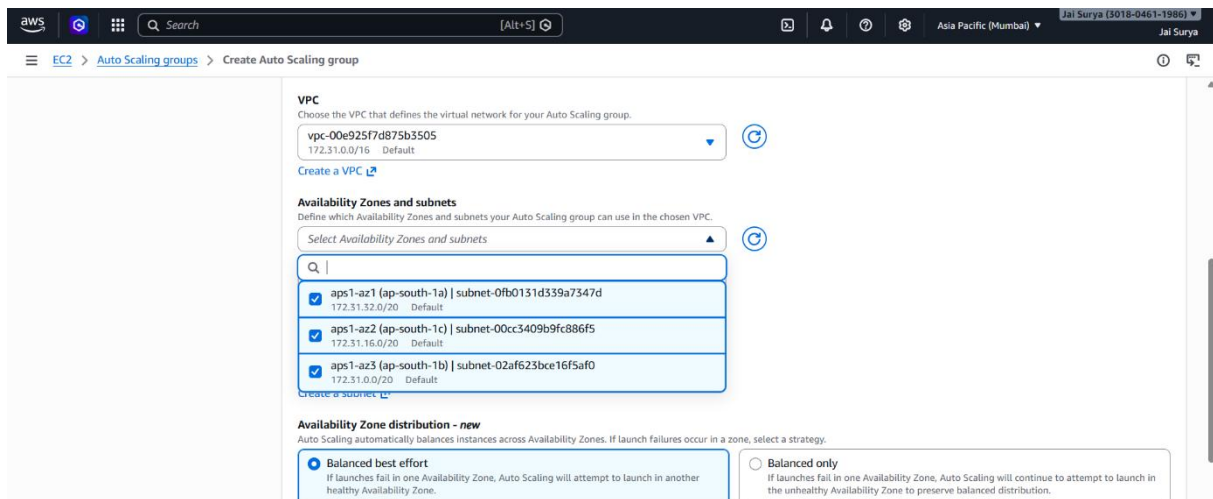
Screenshot: Configure Auto Scaling Group

- Enter Auto Scaling group name
- Select the created launch template



Screenshot: Select Network

- Select all Availability Zones
- Select all subnets



Screenshot: Configure Load Balancer

- Choose No load balancer

Step 1 Choose launch template
Step 2 Choose instance launch options
Step 3 - optional **Integrate with other services**
Step 4 - optional Configure group size and scaling
Step 5 - optional Add notifications
Step 6 - optional Add tags
Step 7 Review

Integrate with other services - optional [Info](#)

Use a load balancer to distribute network traffic across multiple servers. Enable service-to-service communications with VPC Lattice. Shift resources away from impaired Availability Zones with zonal shift. You can also customize health check replacements and monitoring.

Load balancing [Info](#)

Use the options below to attach your Auto Scaling group to an existing load balancer, or to a new load balancer that you define.

Select Load balancing options

- ☒ **No load balancer**
Traffic to your Auto Scaling group will not be fronted by a load balancer.
- ☐ **Attach to an existing load balancer**
Choose from your existing load balancers.
- ☐ **Attach to a new load balancer**
Quickly create a basic load balancer to attach to your Auto Scaling group.

VPC Lattice integration options [Info](#)

To improve networking capabilities and scalability, integrate your Auto Scaling group with VPC Lattice. VPC Lattice facilitates communications between AWS services and helps you connect and manage your applications across compute services in AWS.

Select VPC Lattice service to attach

- ☒ **No VPC Lattice service**
VPC Lattice will not manage your Auto Scaling group's network access and connectivity with other services.
- ☐ **Attach to VPC Lattice service**
Incoming requests associated with specified VPC Lattice target groups will be routed to your Auto Scaling group.

Screenshot: Set Capacity

- Enter desired capacity as 2 (Desired is the number of instances to run)
- Enter minimum capacity as 1 (Minimum capacity is the least number of instances)
- Enter maximum capacity as 4 (Maximum capacity is the limit of instances)
- Click Create Auto Scaling group

Step 2 Choose instance launch options
Step 3 - optional Integrate with other services
Step 4 - optional **Configure group size and scaling**
Step 5 - optional Add notifications
Step 6 - optional Add tags
Step 7 Review

Group size [Info](#)

Set the initial size of the Auto Scaling group. After creating the group, you can change its size to meet demand, either manually or by using automatic scaling.

Desired capacity type
Choose the unit of measurement for the desired capacity value. vCPUs and Memory(GiB) are only supported for mixed instances groups configured with a set of instance attributes.

Units (number of instances) ▼

Desired capacity
Specify your group size.

Scaling [Info](#)

You can resize your Auto Scaling group manually or automatically to meet changes in demand.

Scaling limits
Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity

Equal or less than desired capacity

Max desired capacity

Equal or greater than desired capacity

Screenshot: Auto Scaling Group Created

- Auto Scaling group created successfully

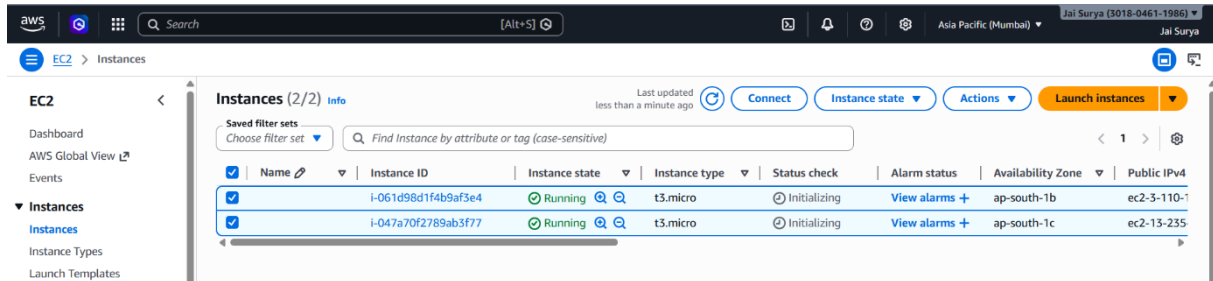
Auto Scaling groups (1/1) [Info](#)

Search your Auto Scaling groups

☒	Name	Launch template/configuration	Instances	Status	Desired capacity	Min	Max	Availability Zones	
☒	auto-scaling	my-template Version Default	0	Updating capacity...	2	1	4	3 Availability Zones	Su

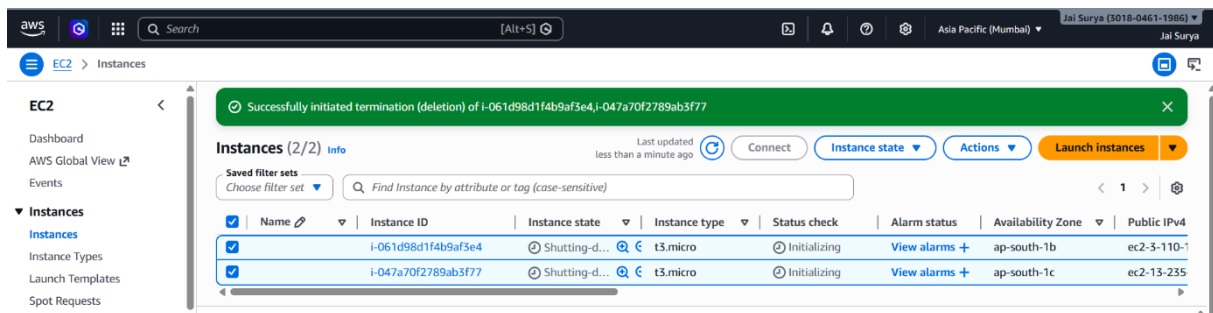
Screenshot: Verify EC2 Instances

- Go to EC2
- Click Instances
- Verify 2 instances are created (desired capacity = 2)



Screenshot: Terminate Instances

- Select both instances
- Click Instance state
- Click Terminate instance

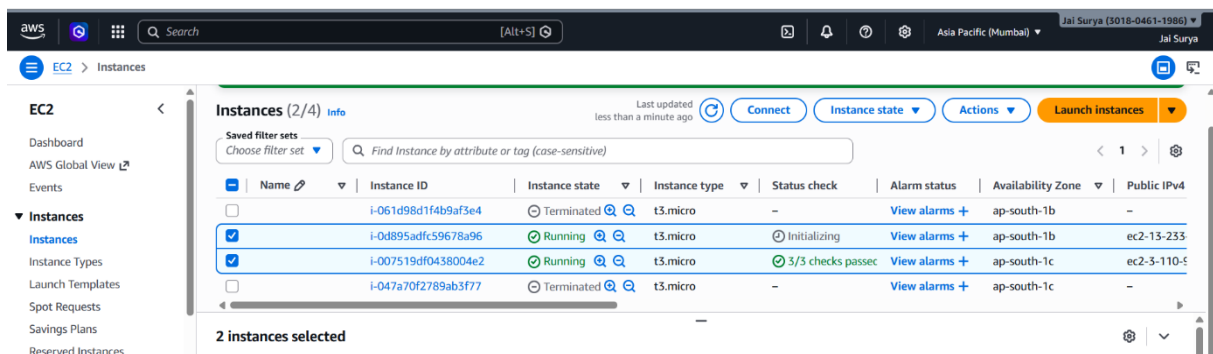


Screenshot: Instances Recreated Automatically

- Two instances are created again automatically
- Instances are in running state

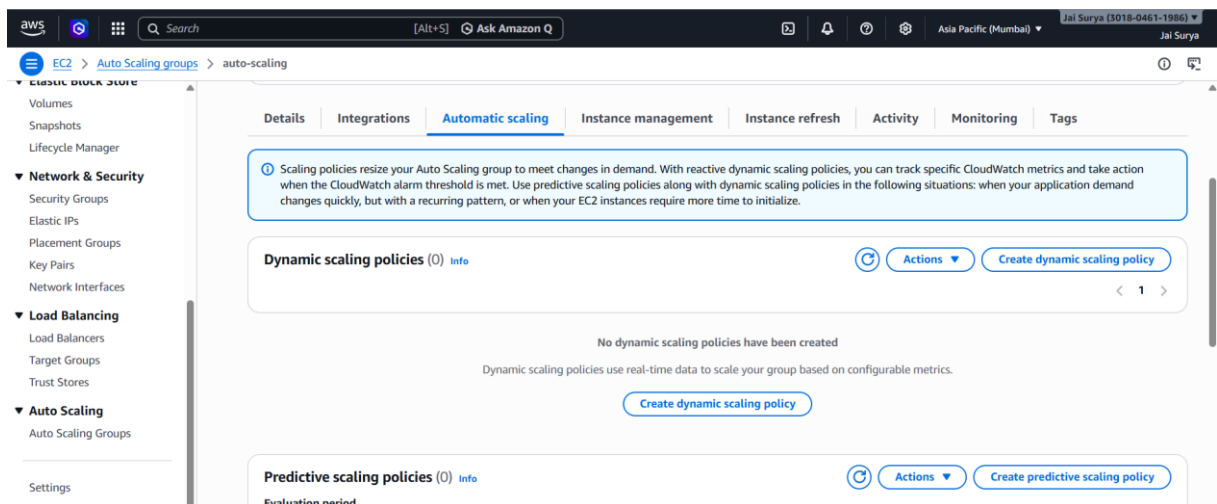
Reason

- Auto Scaling Group maintains desired capacity (Desired capacity is set to 2)
- When instances are terminated, ASG launches new instances automatically



Screenshot: Create Scaling Policy

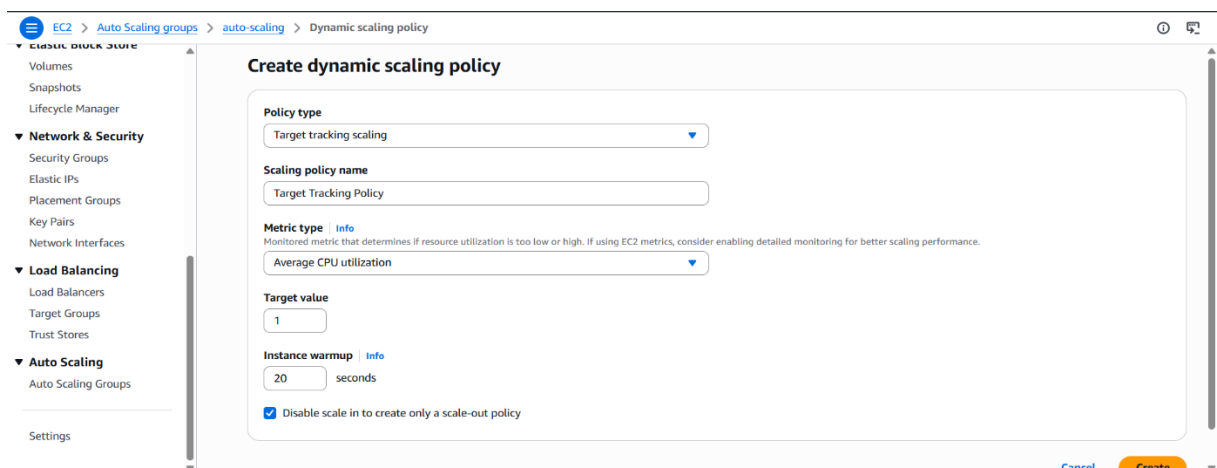
- Go to EC2
- Click Auto Scaling Groups
- Select the created Auto Scaling group
- Go to Automatic scaling
- Click Create dynamic scaling policy



Screenshot: Configure Scaling Policy

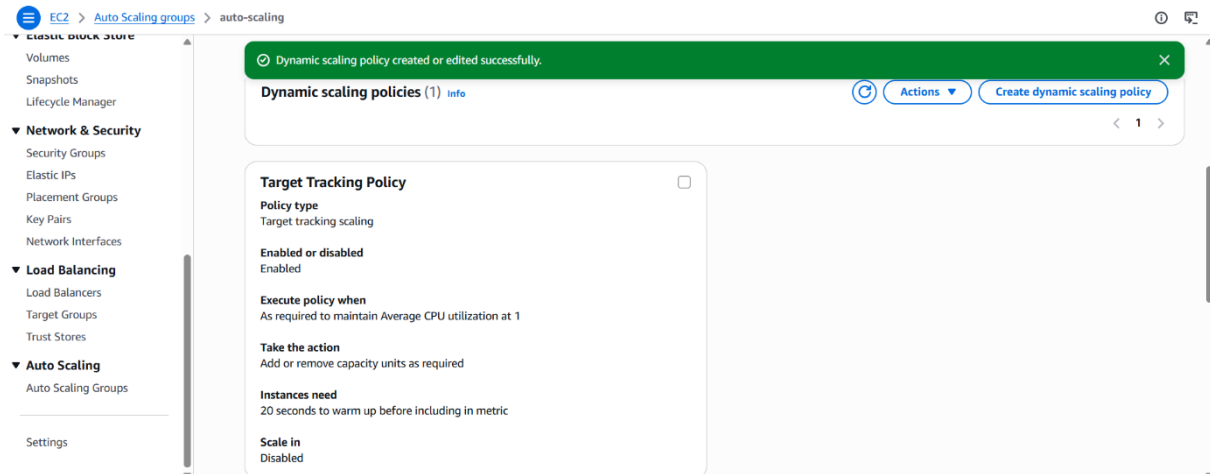
- Select metric as average CPU utilization
- Enter target value as 1
- Set instance warmup as 20 seconds
- Tick disable scale in (only scale out)

Target value is the CPU percentage Auto Scaling tries to maintain .If CPU goes above target value, new instances are added



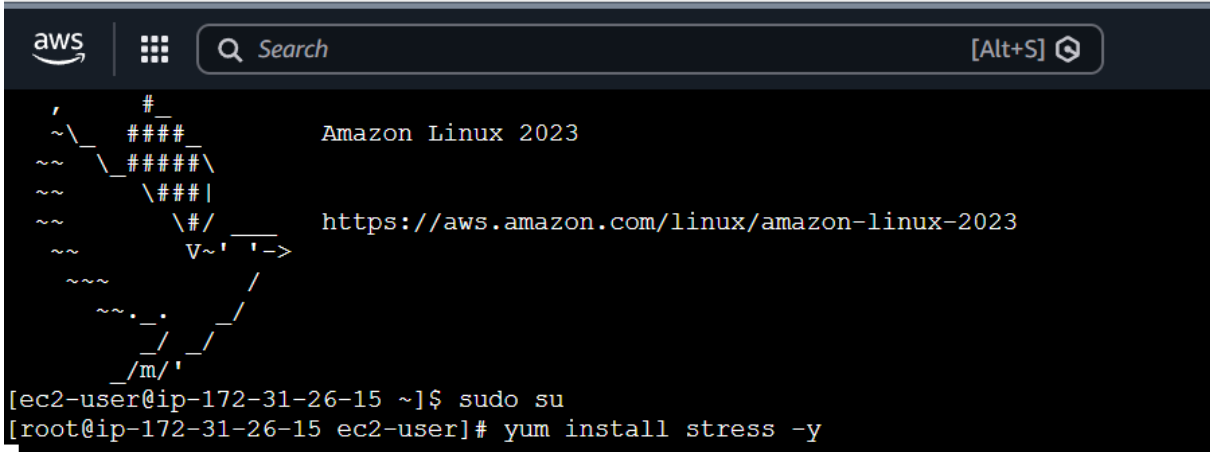
Screenshot: Scaling Policy Created

- Scaling policy created successfully



Screenshot: Connect and Install Stress

- Select both instances
- Click Connect
- Run **sudo su**
- Run **yum install stress -y** on both instances



Screenshot: Increase CPU Utilization

- Run **stress --cpu 4 --timeout 300** on both instances
- This creates fake CPU load to increase utilization

```
Installed:
  stress-1.0.7-2.amzn2023.0.1.x86_64

Complete!
[root@ip-172-31-26-15 ec2-user]# stress --cpu 4 --timeout 300
```

Screenshot: Instances Scaled Out

- Two new instances are added automatically
- Total instances increased after CPU load

The screenshot shows the AWS Management Console interface for EC2 instances. The left sidebar contains navigation links for EC2, Dashboard, AWS Global View, Events, and various instance types. The main content area displays the 'Instances' page with a filter set to 'Running'. A table lists 4 instances, all in a 'Running' state. The first two instances are selected. The table columns include Name, Instance ID, Instance state, Instance type, Status check, Alarm status, Availability Zone, and Public IPv4. The bottom of the table shows '2 instances selected'.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
☒	i-003ef8c2ca7c04ccd	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-1a	ec2-13-235-
☒	i-0d895adfc59678a96	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-1b	ec2-13-233-
☐	i-007519df0438004e2	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-1c	ec2-3-110-
☐	i-097b7c35e9ceb8064	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-1c	ec2-13-203-

2 instances selected